

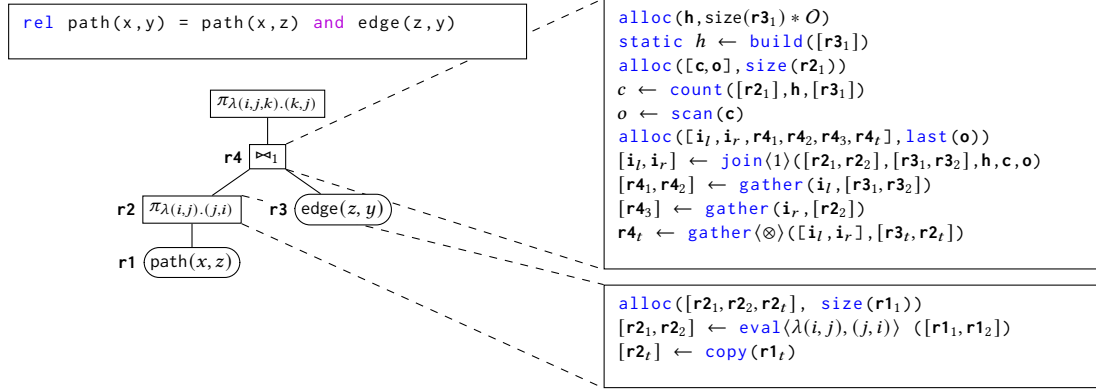


Figure 6. In this example, we compile a part of the rule shown in Figure 3c (line 6). The code block on the top shows the Datalog rule, while bottom-left illustrates the abstract syntax tree of the RAM program compiled from it. We expand the nodes **r2** and **r4** on the right to show their low-level APM code. We denote with O a configurable parameter that determines the size of the hash table used in the join implementation.

to track provenance correctly in joins, where the provenance of each output fact is the product of the provenance of the input facts. A concrete usage of lowering join to APM can be seen in Figure 6, which features the compilation of a join over two binary relations.

3.4 Evaluating APM

Once an APM program is compiled, it is executed in a least fix-point iteration manner. Note that the program executes continuously, updating the database each time, until no new facts are discovered. To make this process efficient, Lobster employs a semi-naive evaluation strategy, which is a variant of the traditional naive evaluation strategy that avoids redundant computation.

Succinctly, semi-naive evaluation involves tracking a frontier of recently discovered facts, and only applying rules to frontier facts. This avoids the redundant computation of applying rules to stale facts that are known a priori to not produce new facts. Concretely, the database is partitioned into three sets of facts: delta facts (those that are computed during the current iteration), recent facts (those that were computed in the previous iteration), and stable facts (all other facts). After each iteration, the recent facts are merged with the stable facts and the delta facts become the recent facts. Importantly, these semantics are codified in translation rules (see Appendix), which express semi-naive evaluation in terms of APM instructions. This means the deduplication of facts and the tracking of recent and stable facts is parallelized on the GPU just like the rest of the computation.

3.5 Provenance Semiring Framework

As discussed prior, Lobster employs a GPU-accelerated provenance semiring framework with 7 implemented semirings covering discrete, probabilistic, and differentiable reasoning.

Specifically, Lobster supports unit, max-min-prob, add-mult-prob, top-1-proof, and the differentiable versions of the probabilistic semirings. Tags in APM are stored as an additional register alongside the column registers. Since the tags may store boolean, floating point, and even complex data structures like dual-numbers and boolean formulae, we need each provenance to specify a fixed size for each tag.

Limitations. Notably, Lobster departs from prior work by not supporting the fully general top- k -proof provenance [20], but just the special case of top-1-proof. This special case tracks just one conjunction of boolean variables for each fact, encoding that fact’s proof set. During disjunction, the provenance picks the more likely of the two proofs by computing the probabilities for each. For conjunction, the provenance merges the two proofs and ensures that no conflict is present. We find that this special case is sufficient for most practical applications and is much more efficient to compute than the general case. Note that in this formulation, the size limit for a proof needs to be specified ahead of time. We set the limit to 300 which is sufficient for all evaluated benchmarks.

4 Optimizations

Section 3 describes the APM language and its value as a principled way to handle Datalog with provenance on GPUs; however, APM shows additional utility as a platform for optimizations. Here, we discuss optimizations from prior works that are easily implementable as APM transformations, as well as novel optimizations enabled by the APM runtime.

4.1 Buffer Reuse and Management

Regardless of evaluation strategy, query evaluation produces lots of temporary data, making allocation performance important. Accordingly, every allocation in an APM program is

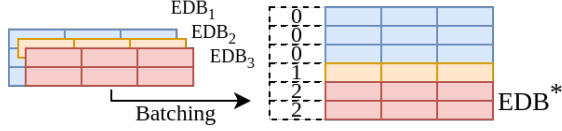


Figure 7. How Lobster incorporates batched input data via sample tagging. Distinct colors represent distinct samples within a batch.

identified by an `alloc` instruction. Due to the lack of branching and looping constructs, each iteration through an APM program produces a fixed number of allocations. This enables two optimizations: arena allocation and buffer reuse.

Arena Allocation. All data in registers is discarded after each iteration of an APM program. This fixed lifetime guarantee makes arena allocation [18] a good fit for APM. This observation makes allocations and deallocations in APM essentially zero-cost, as allocation is reduced to bumping a pointer and deallocation is a no-op.

Buffer Reuse. When arena allocation is impossible due to memory limits, allocation cost can still be amortized via buffer reuse. Prior work on GPU Datalog execution made a similar discovery [44], noting that it was expensive to allocate a specific buffer required for merging relations after each iteration. To address this, the authors over-allocate that buffer initially and reuse it across iterations. Since each APM buffer is identified by an `alloc` instruction, it is trivial to implement this optimization in APM not just for a specific buffer, but for each buffer in the program. This optimization is surprisingly effective, since the size of each register is strongly correlated with its size on the previous iteration.

4.2 Hash Index Reuse via Static Registers

Using a hash-based join is a boon for GPU acceleration, but building a hash index during each iteration of the fix-point loop negatively impacts performance. To resolve this, we observe that frequently at least one input to a join is an EDB relation, which is constant across iterations. Formally, Datalog programs are said to be “linear recursive” if each join has at most one IDB input, and we find that nearly all programs we consider are linear recursive. In these cases, we can build a hash index during the first iteration of the fix-point loop and reuse it during successive iterations.

To realize this, we introduce the concept of static registers, modeled after the `static` keyword in C. Static registers are initialized once and retain their values across iterations. Marking the result of `build` as `static` when compiling join ensures that the hash index is reused across iterations.

4.3 Batched Evaluation

A key component of deep learning is grouping samples into *batches* of samples that can be processed by the model in a single pass. To truly integrate Lobster with deep learning as an end-to-end neurosymbolic tool, it should be aware of batching and able to process batches effectively. Surprisingly, batching is a straightforward extension of the existing semantics. Given a program $\bar{\phi}$ evaluated against a batch of three databases F_1 , F_2 , and F_3 , we seek a database F^* such that evaluating $\bar{\phi}$ against F^* provides equivalent information to evaluating $\bar{\phi}$ against F_1 , F_2 , and F_3 separately.

We can construct F^* without adding new constructs to APM or RAM by taking our APM columnar representation of a table and adding a new register r_s to the front of the table that records the sample id. Tables are now represented as a register pack $[r_s, r_1, \dots, r_n, r_t]$, including the sample id, the columns, and the provenance semiring tags. After execution, each IDB fact will have a sample tag which can be used to disambiguate results into per-sample databases that are returned to the user. This is illustrated in Figure 7.

Some desirable ramifications that naturally arise from this framing of batching are (1) facts from separate batches cannot be joined together, so long as the width of each join operator is extended by one to include the batch tag; (2) parallelizing over each element of the batch is implicit, as the runtime already parallelizes over the rows of a relation; and (3) the additional memory footprint is minimal, since batches are seldom larger than 256 samples, meaning sample tags only take one byte of memory per row.

5 Implementation

We build Lobster with a mixture of Rust, C++, and CUDA, reusing the front-end and query planner of Scallop to limit the scope of implementation. Lobster comprises approximately 2,000 new lines of Rust code and 9,000 new lines of CUDA/C++. We now discuss implementation details that fall outside the scope of the theory of Lobster’s core compiler and runtime, yet are of practical interest and importance for implementing Lobster.

5.1 Hash Table Design

Crucial to Lobster’s GPU-accelerated join algorithm is the existence of a lock-free, GPU hash table supporting parallel insertions and lookups. While many implementations are possible, ours is inspired by previous work in [38]. Namely, we adopt open-addressing with linear probing for collision resolution, enabling a contiguous memory representation with no indirection. Unlike prior work, Lobster supports reasoning over relations with arbitrary width, so rather than storing fact data directly in the hash table, we build hash *indices* that map back to a row of the source table. While this necessitates an additional random memory access to resolve

collisions, it decouples the time and space complexity of the join from the width of the input relations.

5.2 Bytecode Interpreter for Expression Evaluation

Projection operations are pervasive in Datalog programs, yet generally account for a small portion of the runtime compared to operators like `join` due to projection’s algorithmic simplicity. Nonetheless, Lobster optimizes the handling of project operations for GPU execution. Specifically, there are two code paths for the implementation of `eval` in APM. Project expressions that permute or subset the columns of the relation can be evaluated as a series of columnar memory copies. Project expressions that contain arithmetic or comparison of tuple elements are compiled to bytecode for a simple stack machine, and each GPU thread executes this bytecode program against one fact with a small fixed-size stack residing in thread-local memory.

5.3 Scheduling Stratum Offloading

Lobster relations start their life in CPU memory and once in GPU memory it is advantageous to continue operating on them with the GPU. To avoid the impact of high-latency CPU-GPU memory transfers, Lobster adopts an intelligent strategy for scheduling data transfers.

Lobster begins by transferring facts to the GPU before the longest-running stratum (identified via a heuristic based on counting recursive joins) and back to the CPU after that stratum. From that longest-running stratum, we expand forwards and backwards in the static data-dependency graph to encompass adjacent strata as well, until the size of the stratum’s inputs and outputs is small. Adopting a min-cut-like approach to GPU scheduling avoids spending excessive time in CPU-GPU transfers.

5.4 Other Forms of Parallelism

The primary parallelism exploited by Lobster is within each relational operator, and there is no parallelism in the execution of separate relational operators (they are executed sequentially). While the idea of parallelizing or pipelining operators is appealing, since Lobster’s design requires very little CPU-GPU data movement, there is no opportunity to overlap data transfer with computation, so no performance improvement is achieved.

6 Evaluation

We empirically evaluate Lobster with the goal of demonstrating how well it performs in both training and inference tasks, and for the latter we explore a range of benchmarks that require differentiable, probabilistic, or discrete reasoning.

In the following sections, we introduce the benchmark tasks (Section 6.1) and the chosen baselines (Section 6.2) and present results in Sections 6.3 and 6.4. All benchmarks are run on a machine with two 20-core Intel Xeon CPUs,

a GeForce RTX 2080 Ti GPU, and 768 GB RAM, with the exception of the discrete benchmarks, which have higher VRAM requirements and were run on a machine with two 24-core Intel Xeon CPUs, 1.5 TB RAM, and an NVIDIA A100 GPU with 80 GB VRAM.

6.1 Benchmarks

We evaluate Lobster on a suite of ten benchmark tasks summarized in Table 2. Since Lobster is built on a flexible framework of provenance semirings, it supports differentiable, probabilistic, and discrete reasoning. Correspondingly, we pick tasks across each of these reasoning modes to better illustrate the tradeoffs inherent in providing this flexibility. The tasks span diverse application domains: natural language processing (CLUTRR), image processing (Pathfinder and HWF), program reasoning (Probabilistic Static Analysis), bioinformatics (RNA SSP), planning (PacMan-Maze), and graph databases (Transitive Closure, Same Generation, and CSPA).

The table describes each task’s input, the functionality of the logic program, the kind of reasoning involved, and the number of rules. The tasks requiring differentiable reasoning are taken from Scallop’s evaluation (although we omit some tasks that do not have an obvious notion of scalability), the probabilistic reasoning tasks are crafted by us inspired by problems from the literature [40, 42], and the discrete reasoning tasks mirror the evaluation of the latest work in this space, FVLog [43].

Notably, our results focus exclusively on performance, with no mention of correctness. This is because in all cases each system under test produces identical results: for differentiable tasks, the model reaches identical accuracy whether Scallop or Lobster is used, and for probabilistic and discrete tasks, the logic programs are identical and therefore produce identical results. We refer curious readers who desire a more thorough discussion of comparing accuracy between pure-neural and neurosymbolic models to the Scallop paper [27], the results of which we partially replicate here with Lobster.

We next briefly describe each of the tasks.

Pathfinder This task is discussed in-depth in Section 2 and requires reasoning over an image to determine if two dots are connected by a sequence of dashes.

PacMan-Maze In this task, a neurosymbolic reinforcement learning agent aims to solve a 2D maze given only an image of the maze. The neural portion executes a CNN to predict enemy locations in the maze, and the symbolic portion plans a safe path to the goal. Our formulation leverages curriculum learning: the agent first learns in a 5-by-5 maze and then moves to a 20-by-20 grid.

HWF The Handwritten Formula (HWF) [24] task requires parsing and evaluating a formula of handwritten digits and operators, given supervision only on the final value. The dataset consists of formulas of varying length, meaning naive parallelism strategies like processing each formula in a batch